Python for GenAI

Vectors

1. Write a Python script that creates two 3-dimensional vectors using NumPy. Perform and print the results of the following operations: addition, subtraction, and dot product.

2. Implement a Python function that calculates the cosine similarity between two vectors. The function should take two vectors as input and return their cosine similarity score.

3. Given a list of word embeddings (e.g., 10 words with 50-dimensional embeddings), write a Python script to visualize these embeddings in 2D using PCA.
   Hint: Use `sklearn.decomposition.PCA` for dimensionality reduction.

4. Write a Python function that computes the Euclidean distance between two vectors. Test this function on random 5-dimensional vectors.

5. Implement a Python function that performs a brute-force search to find the top 3 most similar vectors to a query vector from a list of vectors using cosine similarity.

6. Load a pre-trained Word2Vec model (e.g., using Gensim) and write a Python script that retrieves and prints the embeddings for five specified words.

7. Write a Python script that uses Facebook's FAISS library to perform a nearest neighbor search. Load a set of 100-dimensional random vectors, build an index, and perform a search to find the nearest neighbors to a query vector.

8. Given a set of 100-dimensional word embeddings, implement a Python script to reduce the dimensionality to 2D using t-SNE and visualize the results.
   Hint: Use `sklearn.manifold.TSNE` for t-SNE.

9. Implement a small-scale vector search application using the Annoy library. Load a set of vectors into Annoy, build an index, and perform a search to retrieve the top 5 closest vectors to a query vector.

10. Write a Python script that integrates with the Pinecone vector database. Load a set of embeddings into Pinecone, perform a search for a given query vector, and retrieve the top 3 results.
    Hint: Use the Pinecone Python client for database interaction.

11. Create a Python function that generates random 50-dimensional vectors and normalizes them to unit length. Store these vectors in a NumPy array and print the first 5 vectors.

12. Write a Python script that uses the cosine similarity function you created earlier to find the most similar pair of vectors in a list of 100 randomly generated 10-dimensional vectors.

13. Implement a Python script that clusters a set of word embeddings using K-Means clustering. Visualize the clusters using 2D PCA projection.

14. Write a Python script to calculate the Manhattan (L1) distance between two vectors. Test this on a set of randomly generated 10-dimensional vectors.

15. Load a set of sentence embeddings using the SentenceTransformers library. Write a Python script to perform a search for the most similar sentences given a query sentence. Use cosine similarity to rank the results.
    Hint: Use `SentenceTransformer` from the `sentence-transformers` package to generate embeddings.